

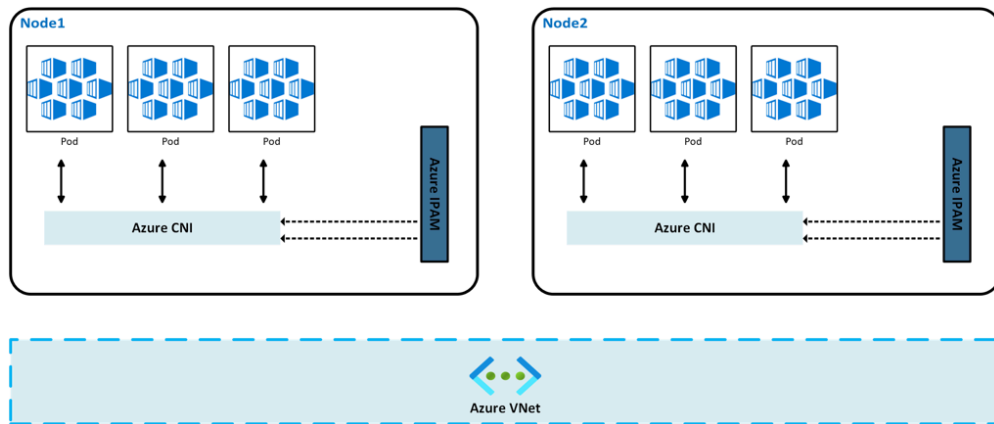
AKS 네트워킹 구조(Azure CNI / Kubenet / CNI Overlay)

AKS 네트워킹은 Pod가 어떤 IP를 가지며, '어떻게 통신하는지'를 결정하는 구조입니다.

Kubernetes는 Pod가 수백, 수천개씩 생겼다가 없어지는 환경이기에 IP 관리 방식이 매우 중요합니다.

AKS는 아래와 같이 총 3가지 네트워크 모드를 지원합니다.

1. Azure CNI



Pod에게 Azure VNet IP를 직접 할당하는 방식으로,

Pod가 Azure VM처럼 VNet 내부에서 동등한 리소스로 취급됩니다.

JH-AKS-DEV Kubernetes Service

이 AKS 클러스터의 모든 Pod 나열 | AKS에서 배포된 특정 Pod 보기 | 이 AKS 클러스터에서 특정 Pod 관리

전환기 | 설정 | 시작 | 중지 | 삭제 | 새로 고침 | 모바일에서 열기 | 피드백 제공

🏠 홈

리소스 그룹 : JH-AKS-DEV

전환 상태 : 실행 중

클러스터의 작업 상태 : 성공

위치 : Visual Studio Enterprise 구독 - MPT

위치 ID : KOREA Central

구독 ID : 85a2eb-8543-4749-81cd-99f1ccca313e

클러스터 관리자 : 할당하려면 여기를 클릭하세요.

태그 (2개) : 태그 추가

시작 | 속성 | 모니터링 | 확장 | 설정

Kubernetes 서비스

일련성 형식 : 클러스터 관리로 이 사용 데이터의 일련성

가상 노드 풀 : 사용 안 함

노드 풀

노드 풀 : 2개 노드 풀

Kubernetes 버전 : 1.33.5

노드 크기 : Standard_D4ds_v5, Standard_D2s_v3

노드 자동 프로비저닝 : 사용 안 함

업그레이드

Kubernetes 버전 : 1.33.5

자동 업그레이드 유형 : 사용 안 함

네트워크

API 서버 주소 : jh-aks-dev-dns-34aapz1hcp.koreacentral.asmiid.io

네트워크 구성 : Azure CNI 노드 서브넷

Pod CIDR : -

서비스 CIDR : 10.0.0.0/16

DNS 서비스 IP : 10.0.0.10

CNI를 100% 지원 : 사용 안 함

네트워크 정책 엔진 : Azure

무엇을 방지할지 : Standard

프라이빗 클러스터 : 사용 안 함

권한 있는 IP 범위 : 사용 안 함

Application Gateway 수인 컨트롤러 : 사용 안 함

Kubernetes 버전 : 1.33.5

API 서버 주소 : jh-aks-dev-dns-34aapz1hcp.koreacentral.asmiid.io

SKU : 기본

가격 책정 계층 : 무료

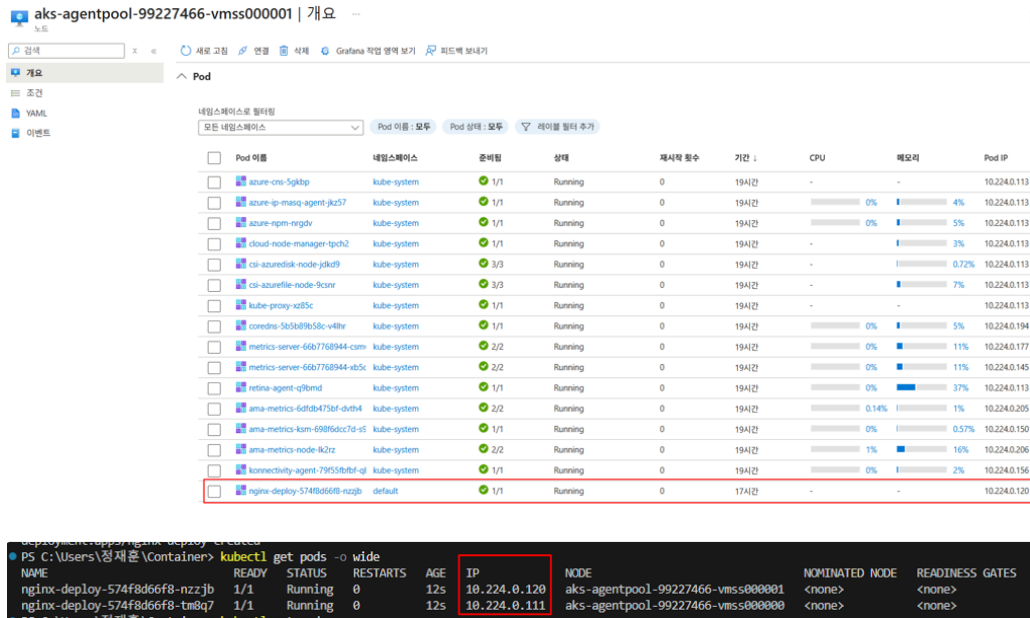
네트워크 구성 : Azure CNI 노드 서브넷

노드 풀 : 2개 노드 풀

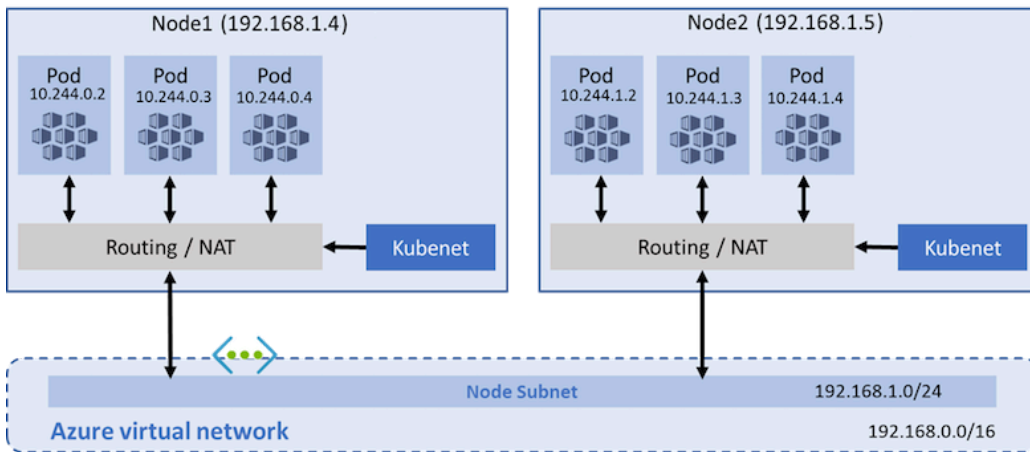
현재의 리지스트리 : 레지스트리의 변경

만든 시간 : December 12, 2023 at 01:17 PM

시작	새로 고침	연결	서버	Grafana 작업 영역 보기	피드백 보내기										
개요	모든 네임스페이스 보기														
구조	POD 이름 : 모든														
YAML	레이블 필터 추가														
이벤트															
	Pod 이름	네임스페이스	준비됨	상태	재시도 횟수	기간	CPU	메모리	Pod IP						
☐	azure-wi-webhook-controller-mu	kube-system	1/1	Running	0	19시간	0%	7%	10.224.0.81						
☐	azure-wi-webhook-controller-mu	kube-system	1/1	Running	0	19시간	0%	6%	10.224.0.95						
☐	eraser-controller-manager-5tc9c	kube-system	1/1	Running	0	19시간	0%	4%	10.224.0.28						
☐	coredns-5d5b89b58c-nk894	kube-system	1/1	Running	0	19시간	0%	5%	10.224.0.108						
☐	coredns-autoscaler-67f9d668db	kube-system	1/1	Running	0	19시간	0%	2%	10.224.0.31						
☐	connectivity-agent-autoscaler-df	kube-system	1/1	Running	0	19시간	0%	3%	10.224.0.93						
☐	azure-csi-nfsn	kube-system	1/1	Running	0	19시간	-	-	10.224.0.4						
☐	azure-ip-masq-agent-t6mgk	kube-system	1/1	Running	0	19시간	0%	3%	10.224.0.4						
☐	azure-ngm-znrzc	kube-system	1/1	Running	0	19시간	0%	5%	10.224.0.4						
☐	cloud-node-manager-lzdd	kube-system	1/1	Running	0	19시간	-	3%	10.224.0.4						
☐	csi-azuredisk-node-wswm6	kube-system	3/3	Running	0	19시간	0.69%	-	10.224.0.4						
☐	csi-azurfile-node-hbz2b	kube-system	3/3	Running	0	19시간	0%	7%	10.224.0.4						
☐	kube-proxy-g8bxr	kube-system	1/1	Running	0	19시간	-	-	10.224.0.4						
☐	retina-agent-jgcvc	kube-system	1/1	Running	0	19시간	0%	39%	10.224.0.4						
☐	ama-metrics-6dfdb475bf-jlgj8	kube-system	2/2	Running	0	19시간	0%	1%	10.224.0.62						
☐	ama-metrics-node-qgjt8	kube-system	2/2	Running	0	19시간	1%	17%	10.224.0.29						
☐	ama-metrics-operator-targets-Sf	kube-system	2/2	Running	2	19시간	0%	1%	10.224.0.34						
☐	connectivity-agent-7f953bfef-ft	kube-system	1/1	Running	0	19시간	0%	2%	10.224.0.89						
☒	nginx-deploy-574fd668-tm8q2	default	1/1	Running	0	19시간	-	-	10.224.0.81						



2. Kubenet

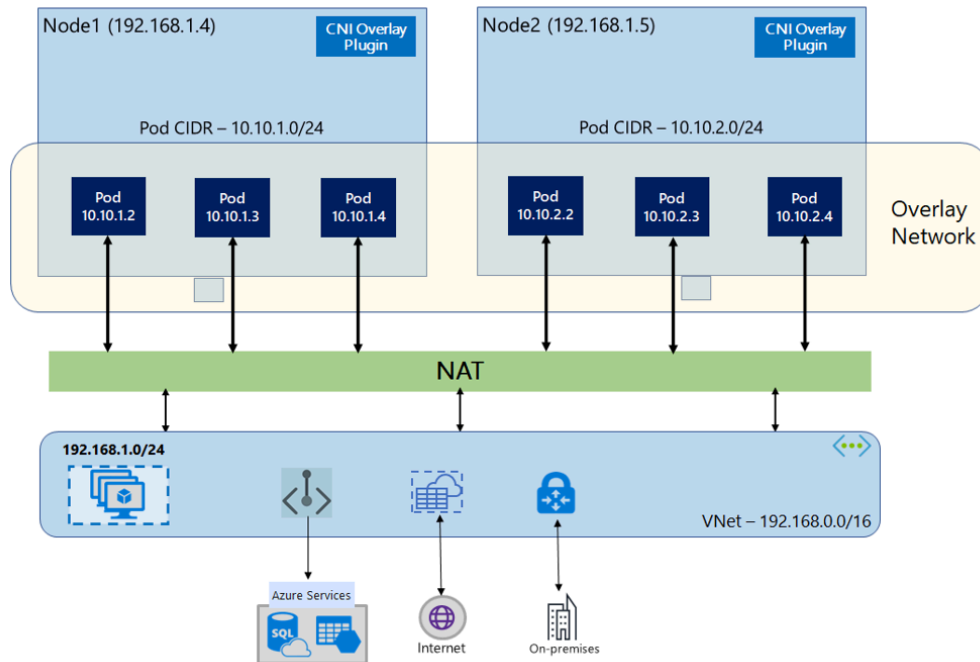


Pod는 Node 내부에서만 유효한 Private IP를 가지고, 외부 통신은 Node의 IP로 NAT해서 나가는 방식입니다.

Kubenet bridge가 Pod ↔ Pod 통신 / Pod ↔ Routing / Pod ↔ 외부(NAT 이후) 등의 트래픽을 노드 내부에서 연결하는 역할을 수행합니다.

※ 단, Azure는 AKS 네트워킹 전략에 따라 Azure CNI Overlay / Azure CNI를 기본으로 밀고 있고, Kubenet은 레거시 옵션 취급을 하기에 기본 UI에서 나타나지 않습니다.

3. Azure CNI Overlay



위 2가지 모델(Azure CNI와 Kubenet)의 장점을 합친 구조인 Azure CNI Overlay에 대해 알아보겠습니다.

Azure CNI Overlay란?

Azure CNI의 장점(기업형 네트워크) + Kubenet의 장점(IP 절약)을 합친 모델입니다.

- Pod에는 Overlay CIDR을 부여합니다.(실제 VNet IP주소를 주지 않습니다.)
- Node ↔ Pod 같은 Overlay 네트워크로 처리합니다.
- 외부 통신은 Azure CNI처럼 처리가 가능합니다.
(NAT은 하지만, 경로와 통제는 Azure 네이티브 네트워크 모델을 따르기 때문입니다.)

즉, IP는 절약하면서도, 기업형 네트워크 기능은 일부 활용 가능합니다.

Azure CNI Overlay에서 등장하는 핵심 용어

• Overlay CIDR

정의 : Overlay CIDR은 Pod 전용으로 사용하는 별도의 IP 주소 범위

-> VNet CIDR : 192.168.0.0/16

Overlay CIDR : 10.10.0.0/16 (Pod 네트워크)

위와 같이 완전히 다른 주소 공간을 사용합니다.

왜 필요한가?

-> 기존의 Azure CNI는 Pod에게 VNet IP를 할당해줬습니다. → VNet Subnet IP가 빨리 고갈 됨

-> Overlay CIDR을 사용하면 VNet의 Subnet IP는 Node에만 사용되고, Pod IP는 Overlay CIDR에서만 할당되기에 IP 고갈 문제가 해결됩니다.

- Overlay Network

정의 : Overlay Network는 Pod들을 위한 가상의 논리적 네트워크입니다.

-> 노드 간 Pod 통신은 Overlay Network위에서 이루어지며, Azure VNet과는 직결되지 않습니다.

-> 비유하자면, Azure VNet = 실제 건물 / Overlay Network = 건물 안의 복도

- Overlay IP (Pod IP)

정의 : Overlay IP는 Pod에게 부여되는 IP로, VNet이 아닌 Overlay CIDR 기반으로 할당되는 IP

→ Pod는 실제 VNet IP와 무관한 주소를 사용하며, VNet 외부에서는 보이지 않습니다

Pod → 외부 : NAT가 필요합니다.

외부 → Pod : 직접 접근이 불가능합니다.

- Overlay Plugin

정의 : Pod 네트워크(IP 할당, 라우팅, 캡슐화 등)를 관리하고 Overlay Network를 구성하는 AKS 내부 네트워크 드라이버입니다.

즉, Overlay 방식에서 Pod IP가 외부(VNet)에 보이지 않기 때문에 Pod 네트워크를 대신 운영해주는 '뇌' 역할을 하는 것이 바로 CNI Overlay Plugin입니다.

CNI Overlay Plugin의 3가지 주요 역할 :

1. Overlay IP 할당 – Overlay CIDR 기반

2. Pod ↔ Pod 통신을 위한 Overlay Routing 처리

→ Pod IP끼리는 VNet 라우팅 테이블에 등록되지 않기 때문에 모든 Overlay 라우팅은 Plugin 내부에서 처리됩니다.

3. Pod → VNet/외부로 나갈 때 NAT 처리

- Overlay Routing

정의 : Overlay Routing은 Pod 간 통신이 Overlay Network 내에서 어떻게 전달되는지 처리하는 방식입니다.

흐름 :

1. Pod가 다른 Pod로 패킷 전송
2. Node의 CNI Overlay Plugin이 패킷을 캡슐화
3. Overlay Network를 통해 대상 Node까지 전달
4. 도착 Node에서 디캡슐화
5. 최종 Pod에 도달

✓ 비유

Pod가 편지를 보내고 싶음 → 편지 주소는 “가짜 주소(Overlay)”임

→ 동네 우체국(VNet)은 모름 → 편지가 안 감

그래서 우체부(Plugin)가 이렇게 함:

“편지를 상자 안에 넣고 상자 밖에는 진짜 주소(Node IP)를 붙임”

진짜 주소끼리 배달되는 통로로 보내버림

도착 Node에서 다시 상자를 열면

원래 편지가 나와서 Pod에게 전달됨

이게 캡슐화 / 디캡슐화

※ kubernetes도 Pod 간 통신은 가능하지만, 항상 Node IP를 경유하는 라우팅/NAT 방식

[Pod1 → Node1 bridge → Node1 IP → Azure VNet 라우팅 → Node2 IP → Node2 bridge → Pod2]

반면에 CNI Overlay는 Pod IP 기반 Overlay 네트워크로 직접 통신합니다.

[Pod1 → Overlay 캡슐화 → Node1 IP → Azure VNet → Node2 IP → Overlay 디캡슐화 → Pod2]

JH-AKS-overlay | 노드 풀

Kubernetes Service

검색

노드 풀 추가

세로 교환

시작

종지

Kubernetes 업그레이드

이미지 업데이트

노드 스케일링

삭제

연결 분석(리거 보기)

레지스터 편집

액션 편집

물결 편집

문제 해결

피드백 제공

노드 풀

노드

노드 자동 프로비전

노드 풀은 매슬라클러스터를 실행할 수 있는 공간입니다. 다양한 유형의 노드 풀을 클러스터에 추가하여 다양한 워크로드를 처리할 수 있고, 기존 노드 풀을 스케일링 및 업그레이드하거나, 더 이상 필요하지 않은 노드 풀을 삭제할 수 있습니다. 각 노드 풀에는 가장 여신에 지원되는 노드가 포함됩니다. [노드 풀에 대해 자세히 알아보기 >](#)

자용 크기 조정 이벤트 0

자용 크기 조정 경고 0

강하가 트리거되지 않음 0

노드 풀 1	프로비전 상태	전환 상태	스케일링 방법	대상 노드 0	준비한 노드 0	자용 크기 조정 상태	모드	운영 체제
node1	성공	실행하는 중	수동	1	1	-	사용자	Azure Linux
agentpool	성공	실행하는 중	수동	1	1	-	시스템	Azure Linux

[illegible]

```

PS C:\Users\정재훈\Container> kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
overlay-demo-5d667f8c5f-jp2bf        1/1     Running   0           47s   10.244.0.112    aks-agentpool-18846361-vmss000000 <none>            <none>
overlay-demo-5d667f8c5f-qh2sz        1/1     Running   0           47s   10.244.1.92     aks-node1-18846361-vmss1          <none>            <none>

```

```

PS C:\Users\정재훈\Container> kubectl exec -it overlay-demo-5d667f8c5f-jp2bf -- sh
/ # ping 10.244.1.92
PING 10.244.1.92 (10.244.1.92): 56 data bytes
64 bytes from 10.244.1.92: seq=0 ttl=62 time=2.038 ms
64 bytes from 10.244.1.92: seq=1 ttl=62 time=1.015 ms
64 bytes from 10.244.1.92: seq=2 ttl=62 time=0.939 ms
64 bytes from 10.244.1.92: seq=3 ttl=62 time=1.176 ms
64 bytes from 10.244.1.92: seq=4 ttl=62 time=1.001 ms
64 bytes from 10.244.1.92: seq=5 ttl=62 time=1.165 ms

```

네트워크 모드	구조	장점	단점
Azure CNI	(1) Node VM에 NIC가 존재합니다. (2) Pod는 Node NIC의 secondary IP 범위를 공유해줍니다. → Pod에 VNet IP를 직접 할당합니다. (3) ∴ Pod도 일반 VNet 리소스처럼 보입니다.	(1) On-prem 및 다른 VNet과 쉽게 통신합니다. (2) NSG, UDR 등 Azure Network 기능들이 Pod에 까지 적용 가능합니다.	(1) Pod 개수가 많아지면 <u>VNet IP 부족 문제가 발생합니다.</u> (2) 1번의 이유로 IP를 많이 소모하면 Subnet 설계가 어렵고 확장 부담이 생깁니다.
Kubenet	(1) Pod는 내부 전용 CIDR을 사용합니다. (2) Node ↔ Pod / Pod ↔ Pod 통신은 Kubenet bridge가 처리합니다. (3) 외부로 나갈 때는 <u>Node IP로 SNAT</u>합니다.	(1) <u>IP 낭비가 없습니다</u> → 큰 규모의 클러스터에 유리 (2) Subnet 관리가 쉽습니다. (3) 비용이 적게 발생합니다.	(1) Pod는 Azure Network(resource)에서 직접 IP로 보이지 않습니다. (2) On-prem 및 VNet 간 트래픽 제약이 많습니다. (3) 정책 관리가 어렵습니다.
Azure CNI Overlay	(1) Pod → Overlay IP (2) Node → VNet IP (3) Overlay 라우팅은 Azure가 자동으로 처리	(1) Pod IP를 많이 써도 Subnet이 부족해지지 않습니다. → <u>IP 낭비가 없습니다.</u>	비교적 최신 기능이기 에 운영사 정책에 따라 제약이 있을 수 있습니다.

	(2) 확장성이 뛰어납니다.	
--	-----------------	--